La persistance des données avec Spring Data JPA et MariaDB

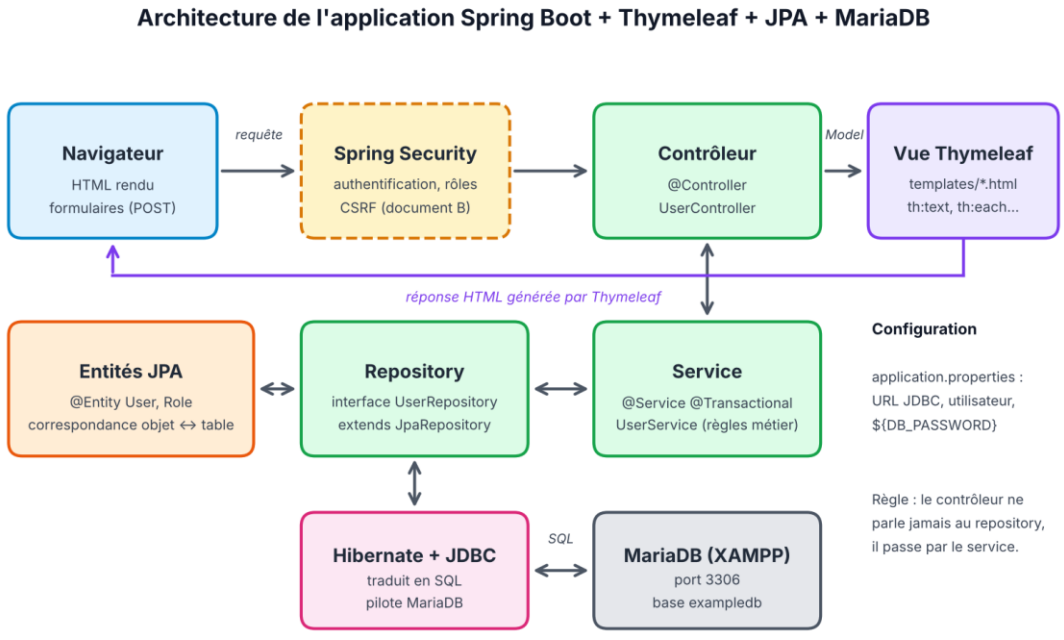
1 DE LA MÉMOIRE À LA BASE DE DONNÉES

Dans le projet précédent, les utilisateurs étaient stockés **en mémoire** : ils disparaissaient à chaque arrêt de l'application. La **persistance** consiste à conserver les données dans un système durable, ici une base de données relationnelle **MariaDB**.

Trois couches logicielles se superposent :

Couche	Rôle
JDBC et le pilote MariaDB	Dialogue de bas niveau avec le serveur de base de données (connexions, requêtes SQL)
JPA (Jakarta Persistence) et son implémentation Hibernate	Correspondance objet-relationnel (ORM) : une classe Java annotée *@Entity* correspond à une table, un objet à une ligne
Spring Data JPA	Génère les accès aux données à partir d'une simple interface (*JpaRepository*) : l'opération *save*, *findById*, *findAll*… n'a plus à être écrite

L'architecture de l'application reste en couches. Seule la couche d'accès aux données change :



Architecture de l'application : du navigateur à la base de données

Le schéma se lit de gauche à droite : la requête traverse (éventuellement) la sécurité, arrive au contrôleur, qui s'adresse au service, puis au repository ; Hibernate et le pilote JDBC dialoguent avec MariaDB. Le contrôleur remplit le *Model*, et la vue Thymeleaf produit la page HTML renvoyée au navigateur. La sécurité est traitée dans le document suivant.

2 PRÉPARER LA BASE MARIADB

2.1 2.1 INSTALLER MARIADB

Deux solutions :

- **Docker** (recommandé, rien à installer sur le poste) :

```
docker run --name mariadb-demo \
  -e MARIADB_ROOT_PASSWORD=mot_de_passe_root \
  -e MARIADB_DATABASE=exampledb \
  -e MARIADB_USER=app \
  -e MARIADB_PASSWORD=mot_de_passe_app \
  -p 3306:3306 -d mariadb:11
```

Cette commande crée la base *exampledb* et l'utilisateur *app*, qui reçoit tous les droits sur cette base. Les mots de passe ci-dessus sont des exemples pour un poste de développement : changez-les.

- **Installation locale** de MariaDB : créez ensuite la base et l'utilisateur comme ci-dessous.
- **XAMPP sous Windows** (si XAMPP est déjà installé sur le poste, MariaDB est fournie avec) :
    1. Dans le panneau de contrôle XAMPP, cliquez sur **Start** en face de **MySQL** (c'est bien MariaDB : XAMPP a conservé l'ancien nom). Apache et Tomcat ne sont pas nécessaires : Spring Boot embarque son propre Tomcat (port 8080). Si le Tomcat de XAMPP est démarré, il occupe ce port : arrêtez-le ou changez *server.port*.
    2. Ouvrez *http://localhost/phpmyadmin* (Apache doit tourner pour cette étape seulement), onglet **SQL**, ou utilisez le bouton **Shell** de XAMPP puis *mysql -u root*.
    3. Exécutez les commandes de la section 2.2 (*CREATE DATABASE*, *CREATE USER*, *GRANT*) puis le script de la section 2.3. Par défaut, *root* n'a pas de mot de passe sous XAMPP : c'est acceptable pour tester sur un poste personnel, mais l'application doit utiliser l'utilisateur dédié *app*.
    4. Le port est 3306, comme dans l'URL JDBC du cours. Si une autre installation MySQL occupe déjà ce port, XAMPP utilisera un autre port (3307, par exemple) : adaptez l'URL.

2.2 Créer la base et l'utilisateur applicatif

Si vous n'utilisez pas Docker, exécutez dans le client *mariadb* (connecté en administrateur) :

```
CREATE DATABASE IF NOT EXISTS exampledb
  CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;

CREATE USER IF NOT EXISTS 'app'@'localhost' IDENTIFIED BY 'mot_de_passe_app';
GRANT ALL PRIVILEGES ON exampledb.* TO 'app'@'localhost';
FLUSH PRIVILEGES;
```

> **Bonne pratique.** L'application ne doit **pas** se connecter avec le compte *root*. Un compte dédié, limité à sa base, réduit les dégâts en cas de faille. *utf8mb4* permet de stocker tous les caractères Unicode (accents, symboles, émojis) ; l'ancien *utf8* de MySQL/MariaDB est limité à 3 octets par caractère.

2.3 Créer la table (script SQL)

```
USE exampledb;

CREATE TABLE IF NOT EXISTS users (
    id       BIGINT AUTO_INCREMENT PRIMARY KEY,
    username VARCHAR(100) NOT NULL,
    email    VARCHAR(100) NOT NULL,
    CONSTRAINT uk_users_username UNIQUE (username),
    CONSTRAINT uk_users_email    UNIQUE (email)
) DEFAULT CHARSET = utf8mb4;
```

Points d'attention :

- La clé *id* est de type ***BIGINT***, pour correspondre au type Java *Long* de l'entité (le script d'origine utilisait *INT*).
- Le nom de la table est ***users***. Nous verrons à la section 5 comment le faire correspondre à l'entité.

2.4 Deux façons de créer les tables

Approche	Principe	Usage
Script SQL (ci-dessus)	On écrit la structure à la main	Contrôle total, base existante
Génération par Hibernate (*spring.jpa.hibernate.ddl-auto*)	Hibernate crée ou adapte les tables à partir des entités	Développement et TP

Dans les deux cas, la structure doit être **cohérente avec les entités**. Le choix de *ddl-auto* est détaillé en section 4.

3 CRÉER LE PROJET

Créez un projet avec Spring Initializr (IntelliJ ou *start.spring.io*) :

Paramètre	Valeur
Project	Maven
Language	Java
Spring Boot	la dernière version stable proposée
Group	*local.epul4a*
Artifact / Name	*springbootdatajpa*
Package name	*local.epul4a.springbootdatajpa*
Java	17 ou supérieur
Packaging	Jar

Dépendances à ajouter :

- **Spring Web** ;
- **Thymeleaf** ;
- **Spring Data JPA** ;
- **MariaDB Driver** ;
- **Validation** (pour le formulaire) ;
- *DevTools* (facultatif).

Un seul pilote. L'énoncé d'origine cochait à la fois *MariaDB Driver* et *MySQL Driver*, ainsi que *Lombok*. Pour MariaDB, un seul pilote suffit : cochez uniquement **MariaDB Driver**. Lombok n'est pas utilisé dans ce document (les accesseurs sont écrits à la main) ; ne l'ajoutez pas si vous ne vous en servez pas.

3.1 LE PILOTE MARIADB AVEC MAVEN

Initializr ajoute la dépendance suivante au *pom.xml* :

```
<dependency>
  <groupId>org.mariadb.jdbc</groupId>
  <artifactId>mariadb-java-client</artifactId>
  <scope>runtime</scope>
</dependency>
```

Il n'y a **pas de *<version>*** : Spring Boot gère lui-même la version du pilote, choisie pour être compatible avec sa propre version. N'indiquez une version que si vous avez une raison précise de la forcer (ce qui est rare).

3.2 STRUCTURE DU PROJET

```
src/main
├── java/local/epul4a/springbootdatajpa
│   ├── SpringbootdatajpaApplication.java
│   ├── controller/UserController.java
│   ├── form/UserForm.java
│   ├── model/User.java
│   ├── repository/UserRepository.java
│   └── service/UserService.java
└── resources
    ├── application.properties
    └── templates
        ├── users.html
        └── add-user.html
```

Le **nom du paquet** doit être strictement le même partout : dans le dossier, dans la ligne `package` de chaque fichier et dans les `import`. La classe `@SpringBootApplication` doit se trouver dans le paquet racine (`local.epul4a.springbootdatajpa`), les autres paquets en dessous, faute de quoi Spring ne les détecte pas.

4 CONFIGURER SPRING BOOT ET JPA

Dans `src/main/resources/application.properties` :

```
spring.application.name=springbootdatajpa

# ================================
# BASE DE DONNÉES
# ================================
spring.datasource.url=jdbc:mariadb://localhost:3306/exampledb
spring.datasource.username=${DB_USERNAME:app}
spring.datasource.password=${DB_PASSWORD}

# ================================
# JPA / HIBERNATE
# ================================
spring.jpa.hibernate.ddl-auto=update
spring.jpa.open-in-view=false
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true
```

Explications :

- ***${DB_PASSWORD}*** : le mot de passe n'est **pas écrit** dans le fichier (qui finit dans le dépôt Git) mais lu dans une variable d'environnement. Sous IntelliJ : *Run* → *Edit Configurations* → *Environment variables* → *DB_PASSWORD=mot_de_passe_app*. Sans cette variable, l'application refuse de démarrer, ce qui est le comportement voulu. *${DB_USERNAME:app}* définit *app* comme valeur par défaut.
- **Le pilote et le dialecte** sont détectés automatiquement à partir de l'URL *jdbc:mariadb://…*. Les lignes *spring.jpa.properties.hibernate.dialect=…MariaDB103Dialect* de l'énoncé d'origine sont à **supprimer** : cette classe est obsolète dans Hibernate 6 et suivants.
- ***spring.thymeleaf.prefix*, *suffix*** **et** ***mode*** sont déjà les valeurs par défaut (*classpath:/templates/*, *.html*, *HTML*) : inutile de les écrire.
- ***spring.jpa.open-in-view=false*** désactive un comportement par défaut qui garde une session de base de données ouverte jusqu'au rendu de la page. Le laisser actif masque des erreurs de chargement paresseux (voir le document sur les rôles) et mobilise une connexion pendant tout le rendu ; Spring Boot affiche d'ailleurs un avertissement au démarrage.
- ***show-sql*** **et** ***format_sql*** affichent les requêtes SQL générées : très utile pour apprendre, à désactiver en production.

Variable d'environnement sous IntelliJ (Windows). *${DB_PASSWORD}* est lu dans l'environnement du processus Java. Dans IntelliJ : *Run* → *Edit Configurations* → *Environment variables*, puis *DB_PASSWORD=mot_de_passe_app*. Après modification d'une variable système Windows, redémarrez IntelliJ.

4.1 LE PARAMÈTRE *DDL-AUTO*

Valeur	Effet	Usage conseillé
none	Aucune action	Production avec un outil de migration
validate	Vérifie que les tables correspondent aux entités, sans rien modifier	Production, ou avec un script SQL
update	Crée les tables et ajoute les colonnes manquantes (ne supprime rien)	**Développement et TP**
create	Supprime puis recrée les tables à chaque démarrage	Tests
create-drop	Comme *create*, et supprime tout à l'arrêt	Tests

Attention. *update* n'est pas un outil de migration fiable : il ne renomme pas les colonnes, ne change pas les types existants et peut laisser la base dans un état inattendu. En production, on utilise *validate* ou *none* avec des migrations versionnées (Flyway ou Liquibase).

4.2 CONVENTION DE NOMMAGE

Par défaut, Spring Boot traduit les noms Java (*camelCase*) en noms SQL (*snake_case*) : une propriété *dateNaissance* est enregistrée dans la colonne *date_naissance*. Si vous branchez l'application sur une base existante, vérifiez que les noms de colonnes suivent cette règle, sinon précisez-les avec *@Column(name = "…")*.

5 L'ENTITÉ *USER*

Une **entité** est une classe Java dont les instances sont enregistrées dans une table.

```java
package local.epul4a.springbootdatajpa.model;

import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.Table;

@Entity
@Table(name = "users")
public class User {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, unique = true, length = 100)
    private String username;

    @Column(nullable = false, unique = true, length = 100)
    private String email;

    protected User() {
        // requis par JPA
    }

    public User(String username, String email) {
        this.username = username;
        this.email = email;
    }

    public Long getId() { return id; }
    public String getUsername() { return username; }
```

```
    public String getEmail() { return email; }

    public void setUsername(String username) { this.username = username; }
    public void setEmail(String email) { this.email = email; }
}
```

Les annotations :

Annotation	Rôle
`@Entity`	Déclare la classe comme entité
`@Table(name = "users")`	Nom de la table. **Indispensable ici** : sans elle, Hibernate utiliserait le nom de la classe (`user`), donc une autre table que `users`. Avec `ddl-auto=update`, il créerait alors une seconde table `user` sans prévenir.
`@Id`	Clé primaire
`@GeneratedValue(strategy = IDENTITY)`	La base génère la valeur (`AUTO_INCREMENT` de MariaDB)
`@Column`	Détails de la colonne : obligatoire (`nullable = false`), unique, longueur

Autres règles à connaître :

- Une entité doit avoir un **constructeur sans argument** (ici `protected`, JPA le trouve) et ne doit pas être `final`.
- Le champ `id` n'a pas de `setId` : il est attribué par la base. Un `id` à `null` signifie « objet pas encore enregistré ».
- Les contraintes `@Column(unique = true)` doivent exister aussi dans la base (script SQL ou `ddl-auto=update`).

6 LE REPOSITORY

```java
package local.epul4a.springbootdatajpa.repository;

import java.util.Optional;

import local.epul4a.springbootdatajpa.model.User;
import org.springframework.data.jpa.repository.JpaRepository;

public interface UserRepository extends JpaRepository<User, Long> {

    Optional<User> findByUsername(String username);

    boolean existsByEmailIgnoreCase(String email);
}
```

Cette interface n'a **aucune implémentation écrite** : Spring Data en génère une au démarrage. `JpaRepository<User, Long>` fournit déjà, entre autres :

Méthode	Effet
`save(user)`	Insère l'entité (si `id == null`) ou la met à jour
`findById(id)`	Renvoie un `Optional<User>`
`findAll()`	Renvoie tous les utilisateurs
`findAll(Pageable)`	Renvoie une page d'utilisateurs, éventuellement triée
`deleteById(id)`	Supprime l'utilisateur
`count()` / `existsById(id)`	Nombre de lignes / existence

6.1 LES MÉTHODES DÉRIVÉES DU NOM

Spring Data analyse le **nom** d'une méthode et génère la requête correspondante :

Méthode	Requête générée (principe)
`findByUsername(String u)`	`… where username = ?`
`existsByEmailIgnoreCase(String e)`	`… where lower(email) = lower(?)`
`findByEmailContainingIgnoreCaseOrderByUsernameAsc(String s)`	Recherche partielle triée
`countByUsernameStartingWith(String p)`	Comptage par préfixe

Mots-clés courants : *And*, *Or*, *Between*, *LessThan*, *GreaterThan*, *Like*, *Containing*, *StartingWith*, *IsNull*, *OrderBy…Asc/Desc*, *IgnoreCase*.

6.2 UNE REQUÊTE EXPLICITE AVEC *@QUERY*

Lorsque le nom devient trop long ou que la requête est complexe, on écrit une requête **JPQL** (un SQL orienté objets, qui parle d'entités et de propriétés, pas de tables et de colonnes) :

```
@Query("select u from User u where lower(u.email) like lower(concat('%', :text, '%'))")
List<User> search(@Param("text") String text);
```

> **Sécurité : injections SQL.** Les méthodes dérivées et les paramètres nommés (*:text*) sont **paramétrés** : la valeur n'est jamais collée dans la requête, ce qui protège contre l'injection SQL. Ne construisez jamais une requête par concaténation de chaînes avec une saisie utilisateur.

7 LE SERVICE

```
package local.epul4a.springbootdatajpa.service;

import java.util.List;

import local.epul4a.springbootdatajpa.model.User;
import local.epul4a.springbootdatajpa.repository.UserRepository;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

@Service
@Transactional(readOnly = true)
public class UserService {

    private final UserRepository userRepository;

    public UserService(UserRepository userRepository) {
        this.userRepository = userRepository;
    }

    public List<User> getAllUsers() {
        return userRepository.findAll();
    }

    public boolean emailExists(String email) {
        return userRepository.existsByEmailIgnoreCase(email);
    }
```

```
    @Transactional
    public User saveUser(User user) {
        return userRepository.save(user);
    }
}
```

Deux améliorations par rapport au code d'origine :

- **Injection par constructeur** à la place de `@Autowired` sur un champ : le champ peut être `final`, les dépendances sont visibles, et la classe se teste sans Spring.
- `@Transactional` : une **transaction** regroupe des opérations qui réussissent ou échouent ensemble. On déclare le service entièrement en lecture seule (`readOnly = true`), puis on surcharge avec `@Transactional` les méthodes qui modifient les données. Une méthode transactionnelle annulée par une exception (`RuntimeException`) annule aussi ses modifications en base.

8 LE FORMULAIRE, LE CONTRÔLEUR ET LES VUES

8.1 L'OBJET FORMULAIRE

On ne lie pas directement l'entité au formulaire : un client malveillant pourrait envoyer un champ `id` et modifier un autre utilisateur. On utilise un objet formulaire distinct, avec les règles de validation :

```
package local.epul4a.springbootdatajpa.form;

import jakarta.validation.constraints.Email;
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.Size;

public class UserForm {

    @NotBlank(message = "Le nom d'utilisateur est obligatoire")
    @Size(max = 100, message = "100 caractères au maximum")
    private String username;

    @NotBlank(message = "L'adresse e-mail est obligatoire")
    @Email(message = "L'adresse e-mail n'est pas valide")
    @Size(max = 100, message = "100 caractères au maximum")
    private String email;

    public String getUsername() { return username; }
    public void setUsername(String username) { this.username = username; }

    public String getEmail() { return email; }
```

```
    public void setEmail(String email) { this.email = email; }
}
```

8.2 LE CONTRÔLEUR

```
package local.epul4a.springbootdatajpa.controller;

import jakarta.validation.Valid;
import local.epul4a.springbootdatajpa.form.UserForm;
import local.epul4a.springbootdatajpa.model.User;
import local.epul4a.springbootdatajpa.service.UserService;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.validation.BindingResult;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.ModelAttribute;
import org.springframework.web.bind.annotation.PostMapping;

@Controller
public class UserController {

    private final UserService userService;

    public UserController(UserService userService) {
        this.userService = userService;
    }

    @GetMapping("/users")
    public String getAllUsers(Model model) {
        model.addAttribute("users", userService.getAllUsers());
        return "users";
    }

    @GetMapping("/add-user")
    public String showUserForm(Model model) {
        model.addAttribute("userForm", new UserForm());
        return "add-user";
    }

    @PostMapping("/add-user")
    public String addUser(@Valid @ModelAttribute("userForm") UserForm form,
                          BindingResult result) {
        if (!result.hasFieldErrors("email") && userService.emailExists(form.g
etEmail())) {
            result.rejectValue("email", "email.duplicate", "Cette adresse e-m
ail est déjà utilisée");
```

```
        }
        if (result.hasErrors()) {
            return "add-user";
        }
        userService.saveUser(new User(form.getUsername(), form.getEmail()));
        return "redirect:/users";   // Post/Redirect/Get
    }
}
```

8.3 *USERS.HTML*

```
<!DOCTYPE html>
<html lang="fr" xmlns:th="http://www.thymeleaf.org">
<head>
  <meta charset="UTF-8" />
  <title>Liste des utilisateurs</title>
</head>
<body>
  <h1>Utilisateurs</h1>

  <p th:if="${#lists.isEmpty(users)}">Aucun utilisateur.</p>

  <table border="1" th:unless="${#lists.isEmpty(users)}">
    <thead>
      <tr><th>ID</th><th>Nom d'utilisateur</th><th>E-mail</th></tr>
    </thead>
    <tbody>
      <tr th:each="user : ${users}">
        <td th:text="${user.id}">1</td>
        <td th:text="${user.username}">nom</td>
        <td th:text="${user.email}">e-mail</td>
      </tr>
    </tbody>
  </table>

  <p><a th:href="@{/add-user}">Ajouter un utilisateur</a></p>
</body>
</html>
```

8.4 *add-user.html*

```
<!DOCTYPE html>
<html lang="fr" xmlns:th="http://www.thymeleaf.org">
<head>
  <meta charset="UTF-8" />
  <title>Ajouter un utilisateur</title>
</head>
<body>
  <h1>Ajouter un utilisateur</h1>

  <form th:action="@{/add-user}" th:object="${userForm}" method="post" novali
date>
    <div>
      <label for="username">Nom d'utilisateur :</label>
      <input type="text" th:field="*{username}" />
      <span th:if="${#fields.hasErrors('username')}" th:errors="*{username}"
            style="color: red;">Erreur</span>
    </div>
    <div>
      <label for="email">E-mail :</label>
      <input type="email" th:field="*{email}" />
      <span th:if="${#fields.hasErrors('email')}" th:errors="*{email}"
            style="color: red;">Erreur</span>
    </div>
    <button type="submit">Enregistrer</button>
  </form>

  <p><a th:href="@{/users}">Retour à la liste</a></p>
</body>
</html>
```

Remarques sur les corrections apportées à ces fichiers : l'attribut `lang="fr "` (avec une espace) et la déclaration `<html lang="fr xmlns:th=…>` (guillemet manquant) de l'énoncé d'origine étaient erronés ; les liens utilisent `th:href="@{…}"` (qui tient compte du chemin de contexte) ; et `th:text` échappe les données affichées.

9 EXÉCUTER ET VÉRIFIER

1. Vérifiez que MariaDB est démarrée (`docker ps` si vous utilisez Docker) et que la variable `DB_PASSWORD` est définie.
2. Lancez l'application. Dans la console, vous devez voir la connexion à la base et, avec `ddl-auto=update`, les requêtes de création de la table si elle n'existe pas.
3. Ouvrez http://localhost:8080/add-user, créez un utilisateur, puis vérifiez la liste sur http://localhost:8080/users.
4. **Vérifiez en base** que la donnée est bien persistée :

```
docker exec -it mariadb-demo mariadb -uapp -p exampledb -e "SELECT * FROM users;"
```

5. **Arrêtez puis relancez l'application** : l'utilisateur est toujours là. C'est la différence essentielle avec le stockage en mémoire.

10 TESTER AVEC UNE BASE EN MÉMOIRE

On évite de dépendre de MariaDB dans les tests automatiques en utilisant **H2**, une base Java qui vit en mémoire le temps du test. Ajoutez dans le `pom.xml` :

```
<dependency>
  <groupId>com.h2database</groupId>
  <artifactId>h2</artifactId>
  <scope>test</scope>
</dependency>
```

Dans `src/test/resources/application.properties` (ce fichier remplace celui de `main` pendant les tests) :

```
spring.datasource.url=jdbc:h2:mem:testdb;MODE=MariaDB
spring.datasource.username=sa
spring.datasource.password=
spring.jpa.hibernate.ddl-auto=create-drop
```

Un test d'intégration qui annule ses modifications à la fin :

```
@SpringBootTest
@Transactional   // chaque test est annulé à la fin : la base reste propre
class UserRepositoryTest {

    @Autowired
    private UserRepository userRepository;

    @Test
    void savedUserCanBeFoundByUsername() {
        userRepository.save(new User("ada", "ada@example.com"));
```

```
        assertThat(userRepository.findByUsername("ada")).isPresent();
    }

    @Test
    void emailExistenceCheckIgnoresCase() {
        userRepository.save(new User("alan", "alan@example.com"));

        assertThat(userRepository.existsByEmailIgnoreCase("ALAN@EXAMPLE.COM")
).isTrue();
    }
}
```

Remarque. Une base H2 en mémoire n'est pas MariaDB : certaines différences de comportement peuvent apparaître (types, fonctions SQL). Pour des tests qui doivent refléter exactement MariaDB, on utilise **Testcontainers**, qui démarre un vrai conteneur MariaDB pour les tests.

11 MIGRER LE PROJET DE GESTION D'UTILISATEURS

Le projet « Gestion d'utilisateurs en mémoire » a été conçu pour qu'on puisse remplacer le stockage **en ne touchant qu'à la couche service**. Voici comment.

11.1 L'ENTITÉ

Dans la classe *User* du projet, ajoutez les annotations JPA, en conservant les règles de validation :

```
@Entity
@Table(name = "users")
public class User {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @NotBlank @Size(max = 50)
    @Column(nullable = false, length = 50)
    private String nom;

    @NotBlank @Size(max = 50)
    @Column(nullable = false, length = 50)
    private String prenom;

    @NotBlank @Email
    @Column(nullable = false, unique = true, length = 100)
    private String email;
```

```
    @NotNull @Past
    @DateTimeFormat(iso = DateTimeFormat.ISO.DATE)
    @Column(nullable = false)
    private LocalDate dateNaissance;   // colonne « date_naissance »

    // constructeurs, accesseurs
}
```

11.2 Le repository

```
public interface UserRepository extends JpaRepository<User, Long> {

    boolean existsByEmailIgnoreCase(String email);

    boolean existsByEmailIgnoreCaseAndIdNot(String email, Long id);
}
```

11.3 Le service

```
@Service
@Transactional(readOnly = true)
public class UserService {

    private final UserRepository repository;

    public UserService(UserRepository repository) {
        this.repository = repository;
    }

    public long count() {
        return repository.count();
    }

    public PageResult<User> findPage(int page, int size, String sort) {
        long total = repository.count();
        int totalPages = (int) Math.max(1, Math.ceil((double) total / size));
        int number = Math.min(Math.max(page, 0), totalPages - 1);

        Page<User> result = repository.findAll(PageRequest.of(number, size, s
ortFor(sort)));
        return new PageResult<>(result.getContent(), number, size, result.get
TotalElements());
    }

    public Optional<User> findById(Long id) {
        return repository.findById(id);
```

```
    }

    @Transactional
    public User create(User user) {
        user.setId(null);   // c'est la base qui attribue l'identifiant
        return repository.save(user);
    }

    @Transactional
    public Optional<User> update(Long id, User data) {
        return repository.findById(id).map(existing -> {
            existing.setNom(data.getNom());
            existing.setPrenom(data.getPrenom());
            existing.setEmail(data.getEmail());
            existing.setDateNaissance(data.getDateNaissance());
            return existing;   // pas de save() : voir « dirty checking » ci-
dessous
        });
    }

    @Transactional
    public boolean deleteById(Long id) {
        if (!repository.existsById(id)) {
            return false;
        }
        repository.deleteById(id);
        return true;
    }

    public boolean emailExists(String email, Long excludedId) {
        return excludedId == null
                ? repository.existsByEmailIgnoreCase(email)
                : repository.existsByEmailIgnoreCaseAndIdNot(email, excludedI
d);
    }

    /** Liste blanche des critères de tri (le paramètre vient de l'URL). */
    private static Sort sortFor(String sort) {
        return switch (sort) {
            case "nom" -> Sort.by(Sort.Order.asc("nom").ignoreCase(), Sort.Or
der.asc("prenom").ignoreCase(), Sort.Order.asc("id"));
            case "dateNaissance" -> Sort.by("dateNaissance", "id");
            default -> Sort.by("id");
        };
    }
}
```

Le contrôleur et les vues ne changent pas.

Deux notions importantes apparaissent ici :

- **Le contexte de persistance et le « dirty checking »** : dans une méthode `@Transactional`, les entités chargées sont *gérées* par JPA. Si on modifie leurs champs, JPA détecte le changement et envoie l'`UPDATE` à la fin de la transaction, sans appel à `save`. C'est ce que fait `update` ci-dessus.
- **La pagination par la base** : `PageRequest` et `Sort` demandent à la base de ne renvoyer que la page voulue, triée (clauses `LIMIT` et `ORDER BY`), alors que la version en mémoire triait et découpait toute la liste. Avec des milliers de lignes, c'est une différence majeure.

11.4 Les données de départ

La version en mémoire chargeait 12 utilisateurs au démarrage. Avec une base, on ne le fait que si la table est vide, et seulement en développement :

```
@Component
class SampleDataLoader implements ApplicationRunner {

    private final UserRepository repository;

    SampleDataLoader(UserRepository repository) {
        this.repository = repository;
    }

    @Override
    public void run(ApplicationArguments args) {
        if (repository.count() > 0) {
            return;
        }
        repository.save(new User("Lovelace", "Ada", "ada.lovelace@example.com
", LocalDate.of(1815, 12, 10)));
        repository.save(new User("Turing", "Alan", "alan.turing@example.com",
 LocalDate.of(1912, 6, 23)));
        // ... et ainsi de suite
    }
}
```

Pour réserver ce chargement au développement, ajoutez `@Profile("dev")` à la classe et lancez l'application avec `--spring.profiles.active=dev`.

11.5 Ce qui change dans la gestion des erreurs

L'unicité de l'e-mail est maintenant garantie à deux niveaux : la vérification dans le contrôleur (message clair) et la contrainte `UNIQUE` de la base (dernier rempart). Si deux requêtes simultanées passent la première vérification, la seconde échoue sur la contrainte avec une `DataIntegrityViolationException` : dans une application réelle, on l'intercepte (par un `@ExceptionHandler`) pour réafficher le formulaire avec l'erreur.

12 Erreurs fréquentes

Symptôme	Cause probable
`Table 'exampledb.user' doesn't exist`	`@Table(name = "users")` manquant : Hibernate cherche une table nommée comme la classe
`Access denied for user…`	Mauvais mot de passe ou variable `DB_PASSWORD` non définie dans la configuration d'exécution
`Communications link failure` / `Connection refused`	MariaDB n'est pas démarrée, ou mauvais port
`Unknown database 'exampledb'`	La base n'a pas été créée
`Schema-validation: wrong column type`	Le type de la colonne ne correspond pas à l'entité (`INT` au lieu de `BIGINT`, par exemple) avec `ddl-auto=validate`
`No qualifying bean of type UserRepository`	La classe de démarrage n'est pas dans le paquet racine, ou une faute de paquet dans un `import`
`Unknown column 'date_naissance'`	Table créée à la main avec un autre nom de colonne que la convention `snake_case`
`LazyInitializationException`	Accès à une relation paresseuse hors transaction (voir le document sur les rôles)
`Access denied for user 'app'@'localhost'` alors que le mot de passe est correct (XAMPP)	Le `GRANT` n'a pas été exécuté, ou l'utilisateur n'existe que pour `localhost` alors que la connexion arrive par `127.0.0.1` : créer aussi `'app'@'127.0.0.1'`
MySQL ne démarre pas dans XAMPP	Le port 3306 est déjà occupé par une autre installation MySQL/MariaDB ; arrêter l'autre service ou changer le port
Accents illisibles	Base créée sans `utf8mb4`
Page inaccessible sur le port 8080	Le Tomcat de XAMPP est démarré et occupe le port
Données qui disparaissent au redémarrage	`ddl-auto=create` ou `create-drop` en développement

13 RÉSUMÉ

- JPA fait correspondre des classes `@Entity` à des tables ; Spring Data JPA génère les repositories à partir d'interfaces.
- Un seul pilote (MariaDB), sans version dans le `pom.xml`, et pas de `root` ni de mot de passe écrit dans le dépôt.
- `@Table(name = "users")` aligne l'entité sur la table ; les types (`BIGINT` pour `Long`) et les noms de colonnes (`snake_case`) doivent correspondre.
- `ddl-auto=update` convient au développement ; en production, `validate` ou `none` avec des migrations.
- Le service est transactionnel (`@Transactional`), reçoit son repository par constructeur et manipule des entités gérées (dirty checking).
- Les formulaires utilisent un objet dédié, jamais l'entité directement ; les requêtes sont paramétrées.
- Le projet en mémoire se migre en modifiant seulement l'entité, le repository et le service.

14 EXERCICES PROPOSÉS

1. Mettre en place MariaDB, lancer l'application et vérifier avec le client que la donnée survit à un redémarrage.
2. Ajouter à `UserRepository` une méthode `findByEmailContainingIgnoreCaseOrderByUsernameAsc` et l'utiliser pour une recherche sur la page de liste.
3. Passer `ddl-auto` à `validate` avec le script SQL de la section 2.3, puis modifier volontairement le type d'une colonne et observer l'erreur.
4. Activer `logging.level.org.hibernate.SQL=debug` et repérer les requêtes générées pour `findAll(PageRequest.of(0, 10, Sort.by("nom")))`.
5. Migrer le projet « Gestion d'utilisateurs en mémoire » vers JPA, et adapter les tests du service pour qu'ils utilisent H2.
6. Ajouter un `@ExceptionHandler` pour `DataIntegrityViolationException` qui réaffiche le formulaire avec un message clair.